

Computación Paralela- Laboratorio 2 - Vectorización

Computación paralela 2021-FAMAF

Cremona Javier
Castillo Marcelo

Introducción

Las computadoras paralelas a menudo se clasifican de acuerdo con un esquema propuesto por Michael Flynn en 1972. En la taxonomía de Flynn, ilustrada en la Tabla 1, hay cuatro clases de paralelismo, que se distinguen sobre la base del flujo de datos e instrucciones: se visualiza una aplicación que se ejecuta en una computadora como una o más secuencias de instrucciones y una o más flujos de datos, y las computadoras se dividen en cuatro clases dependiendo de si se permiten múltiples flujos de datos o instrucciones. Tres de estas cuatro clases de arquitecturas informáticas paralelas son de interés para la tecnología cuántica, aplicaciones químicas y se describen a continuación.

Instrucción única, datos únicos

La arquitectura de instrucción única, datos únicos (SISD) tiene una instrucción única por flujo y un solo flujo de datos. El flujo de instrucciones y el flujo de datos interactúan, y el flujo de datos está completamente determinado por aquellas instrucciones en el flujo de instrucciones que pertenecen al movimiento de datos.

Instrucción única, datos múltiples

La computación científica a menudo implica realizar la misma operación en diferentes datos. Una forma natural de hacer esto es proporcionada por la instrucción única, Arquitectura de datos múltiples (SIMD), en la que un único flujo de instrucciones simultáneamente opera en múltiples flujos de datos.

Los microprocesadores modernos tienen extensiones que permiten instrucciones únicas para realizar operaciones idénticas en varios datos. Los ejemplos incluyen las extensiones de transmisión SIMD de Intel (SSE, AVX) y Motorola® AltiVec™ utilizado en versiones del procesador IBM POWER™. Las computadoras SIMD a gran escala como la CM-2 no han sobrevivido.

Instrucción múltiple, datos múltiples

La arquitectura de múltiples instrucciones y múltiples datos (MIMD) permite múltiples secuencias de instrucciones para interactuar simultáneamente con su propia secuencia de datos.

Flynn's taxonomy for parallel computers

SISD	Single-instruction, single-data
SIMD	Single-instruction, multiple-data
MISD	Multiple-instruction, single-data
MIMD	Multiple-instruction, multiple-data

Tabla 1: taxonomía de Flynn

Desarrollo del Laboratorio 2:

Para más detalle, ver la presentación: <https://youtu.be/xeBgcsJLczs>

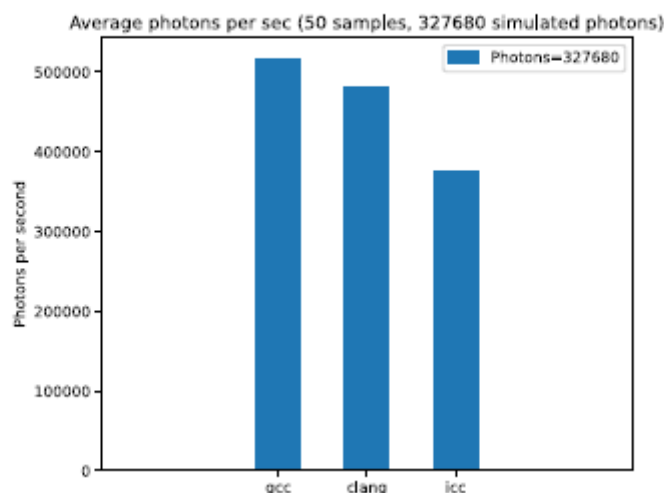
Correcciones al Laboratorio 1:

Inicialmente, antes de empezar con la vectorización, se realizaron dos modificaciones en comparación al laboratorio 1:

- Reemplazo del PRNG por PCG: este ultimo es mas rapido para generar números aleatorios que rand48 de la librería boost de C++.
- Se usa el compilador icc de Intel. En el lab 1 usamos gcc y Clang.

Los resultados del lab 1 en JupiterAce arrojan los siguientes resultados

Optimizaciones: O3, -march=native, -ffast-math, FDO



En la gráfica se puede ver que se obtienen mejores resultados con gcc. Para icc se usaron las banderas equivalentes a las mencionadas.

Vectorización:

Hasta el momento solo teníamos una implementación secuencial en la simulación de fotones, es decir, los fotones se simulaban uno a uno hasta completar la totalidad. Ahora se simularon varios fotones en paralelo y para ello se emplean instrucciones vectoriales. Una modificación inicial al generador PCG: se tenía como array of Struct y a partir de ahora como Struct of array.

Los problemas con los que nos encontramos al querer simular varios fotones en simultáneo son:

- los fotones no terminan al mismo tiempo, a algunos fotones les puede llevar más iteraciones.
- hay bucles para los que no se sabe cuando van a terminar. En particular, en nuestro código, hay un bucle donde se pide un par de números aleatorios y el mismo no finaliza hasta que estos números estén dentro de un círculo unitario.

Estos problemas complican la auto-vectorización por parte del compilador.

Se probaron distintas implementaciones independientes:

1. Implementamos todo en ispc.
2. Modificamos el método `photon()` buscando que el compilador lo auto-vectorice. Pedimos números aleatorios antes de usarlos en el bucle principal (lo llamamos *rand-offline*),
3. Modificamos el método `photon()` buscando que el compilador lo auto-vectorice. Pedimos números aleatorios cuando se necesitan dentro del bucle principal (lo llamamos *rand on-demand*).

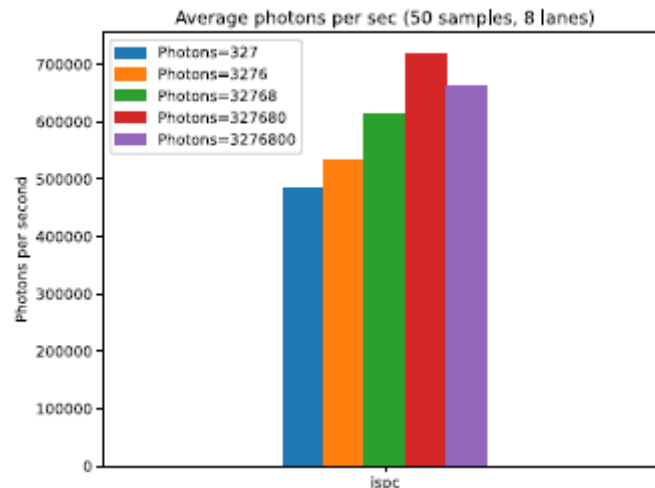
Caso 1: ISPC

Se implementó el generador de números aleatorios vectorizado en *ispc*. Movimos el for que cuenta la cantidad de fotones dentro de la nueva implementación de `photon()`. Dado que hay fotones que finalizan antes que otros, no queremos que ese carril esté inactivo hasta que termine de simularse un pack de fotones. Para ello cuando un fotón finaliza reseteamos ese carril llevando a las variables a los valores iniciales y llevamos un contador que lleva la cuenta de los fotones simulados.

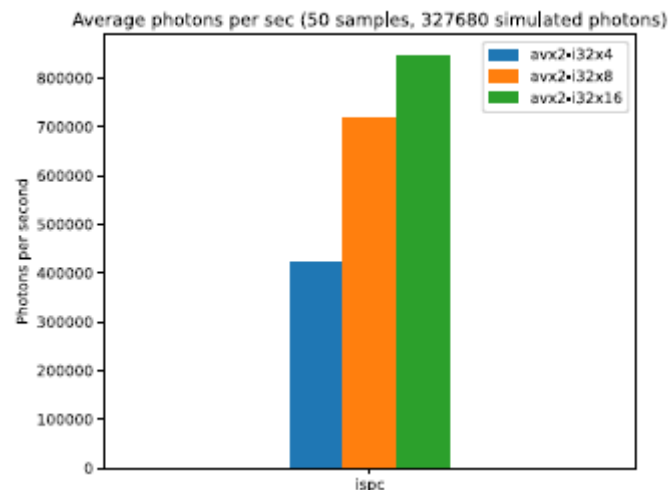
El resto del código permanece igual que en C, solo que los arreglos `heat` y `heat2` van a ser *varying* dado que `shell` puede tomar el mismo valor en dos carriles y tendríamos una *race condition*. Para resolver este problema tendremos un arreglo por cada carril, juntando todo al final.

Resultados con distintos tamaños de problema y distinta cantidad de lanes:

Realizando una gráfica de cantidad de fotones por segundos en ISPC con 8 lanes se puede observar que llegamos casi a 700 K fotones por segundo.



Usando 4, 8 y 16 carriles vemos que para 16 se obtiene por encima de 800 K fotones por segundo.



Caso 2: rand-offline

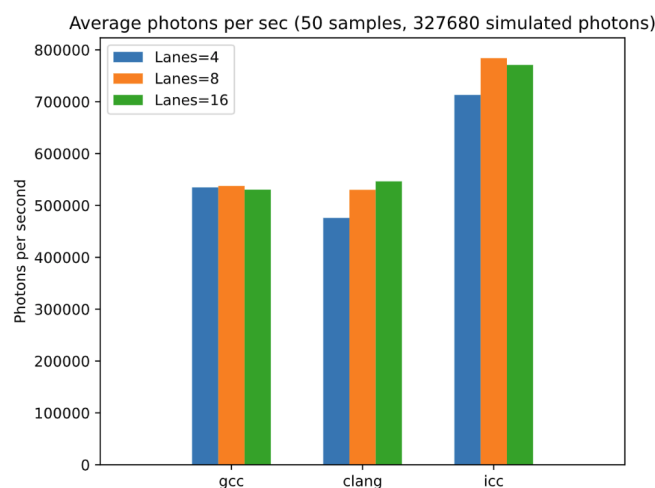
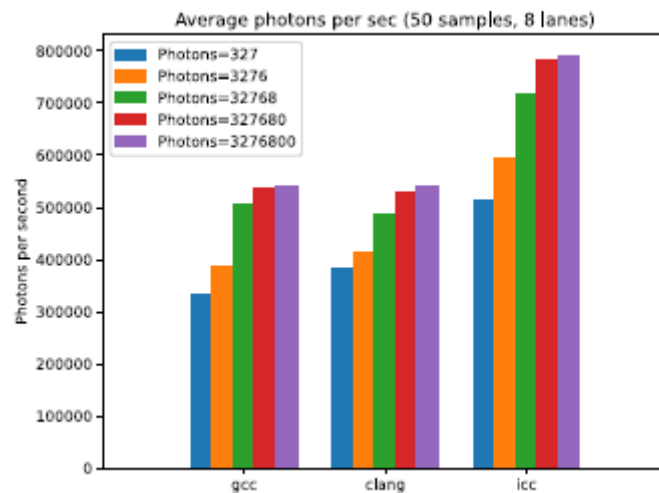
Convertimos las variables escalares del método *photon()* en arreglos de N elementos y movimos el for que cuenta la cantidad de fotones dentro de la nueva implementación de *photon()*. Aquí N representaría los carriles. Hicimos pruebas para 4, 8 y 16 carriles (es decir, se probó con tamaño 4, 8 y 16 para los arreglos que reemplazan a las variables escalares). También se vectorizó el generador de números aleatorios.

En lugar de frenar un carril cuando finaliza la simulación de un *photon()*, reseteamos ese carril, y llevamos la cuenta de cuántos fotones llevamos simulados. Los números aleatorios que se van a usar en una iteración del bucle principal se piden antes de iniciar una nueva iteración vectorizada, en un bucle previo. Así evitamos frenar los carriles dentro del bucle principal y movemos un loop do-while fuera del mismo. Además, guardamos el calor en arreglos temporales de tamaño igual a la cantidad de carriles. En cada posición guardamos el calor obtenido para ese carril.

Al final de cada iteración (vectorizada), en un loop aparte acumulamos el calor en la capa correspondiente en el array global y luego reseteamos los arrays temporales.

Resultados con distintos tamaños de problema y distinta cantidad de lanes:

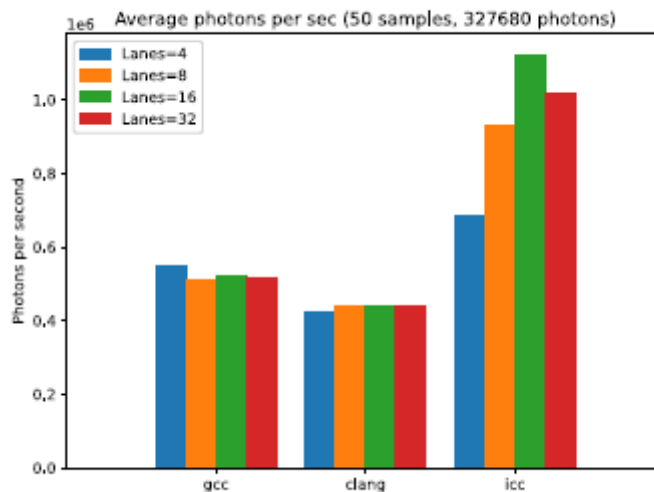
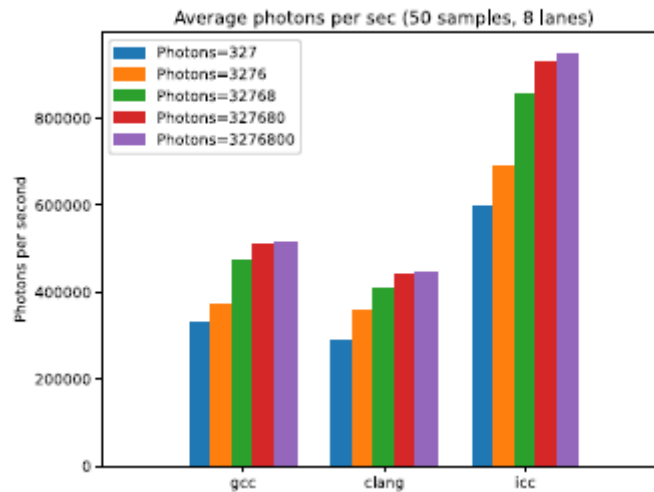
Los resultados en Jupiter Ace se muestran a continuación



Caso 3: rand-ondemand

Similar al Caso 2, excepto que en esta ocasión los números aleatorios se piden dentro del bucle. Llevamos una máscara durante toda la ejecución del bucle vectorizado. Si el par de números aleatorios (que se pedía anteriormente en el Lab. 1 iterando en un do-while) no cumple con la condición de pertenecer al círculo unitario, se detiene únicamente ese carril usando la máscara. Así, esto nos permite remover el inner loop do-while para el cual no se sabía cuántas iteraciones realizaba. En la próxima iteración vectorizada, ese carril vuelve a pedir un par de números aleatorios y si ahora cumple la condición de pertenecer al círculo unitario, avanza. Caso contrario, se repite lo mismo.

Resultados con distintos tamaños de problema y distinta cantidad de lanes:



Discusión

El mejor resultado obtenido fue con icc y con la implementación 3, logrando casi 1.2 M de fotones por segundo. Vale remarcar que solo icc pudo sacar provecho de la vectorización de la implementación 3. Para clang y gcc, prácticamente no se obtuvieron mejoras respecto a la implementación del Laboratorio 1. De hecho, para clang y gcc es un retroceso, ya que el código se volvió más complejo y la performance es prácticamente la misma. En cuanto a icc, se pasó de menos de 400 K fotones por segundo (en el Laboratorio 1 en JupiterAce) a más de 1.1 M. Vimos que icc es más “astuto” para autovectorizar. Los reportes de vectorización de icc son bastante completos, y se puede aprender mucho del código leyéndolos. Otra cuestión a subrayar es que para icc tanto en Sandy Bridge (PC de escritorio en la que realizamos algunas pruebas) como en Broadwell, se observaron mejores resultados con 16 lanes que con 8, aunque ninguna de las dos arquitecturas cuenta con AVX512. Recordar que para las implementaciones 2 y 3, con “lanes” nos

referimos al tamaño de los arreglos utilizados para reemplazar las variables escalares.

En resumen, probamos diferentes implementaciones, en las que el código de ISPC es el más legible y para los casos 2 y 3 se requirió mayor esfuerzo del programador. A futuro, se podría implementar una aproximación para la función log, ya que perf sigue reportando que es una función cara.

Otras cuestiones a remarcar

Para finalizar, algunos detalles a mencionar. En Sandy Bridge (SB), prácticamente no hay diferencia entre los casos 2 y 3. Cabe destacar también que en SB, icc vectoriza loops con O2 que con O3 no puede vectorizar.